Practical no 10: Performing INSERT, UPDATE, DELETE and SELECT Operations using Python GUI and MySQL

Question

Design and develop a Python GUI application to connect with a MySQL database and perform the following operations on student records:

1. Insert new student records using the `INSERT` command.
2. Retrieve and display student records using the `SELECT` command.
3. Modify existing student records using the `UPDATE` command.
4. Delete student records using the `DELETE` command.
5. Provide suitable GUI widgets for entering and displaying the data.

Step 1: Create the MySQL Database

Open **MySQL Workbench** and execute:

```
CREATE DATABASE college_db;
```

Select the database:

```
USE college_db;
```

Step 2: Create the Table

Execute the following SQL command:

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    course VARCHAR(100),
    email VARCHAR(100)
);
```

Check the table:

```
SELECT * FROM students;
```

Initially, the table will be empty.

Step 3: Install MySQL Connector for Python

Open Command Prompt or Terminal and execute:

```
pip install mysql-connector-python
```

The connector allows Python to communicate with MySQL.

Step 4: Create Python Project

Create a project folder:

```
StudentMySQL
```

Create a Python file:

```
student_mysql.py
```

Project structure:

```
StudentMySQL
│
└── student_mysql.py
```

Step 5: Import Required Modules

Add:

```
import tkinter as tk
from tkinter import messagebox
import mysql.connector
```

Step 6: Connect Python with MySQL

Add the following function:

```
def connect_database():

    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="your_password",
        database="college_db"
```

```
    )

    return connection
```

Important

Replace:

`"your_password"`

with your MySQL root password.

For example:

`password="root123"`

Step 7: Implement INSERT

Create the following function:

```
def insert_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "" or name == "" or course == "" or email == "":
        messagebox.showwarning(
            "Warning",
            "Please enter all details."
        )
        return

    connection = connect_database()
    cursor = connection.cursor()

    query = """
    INSERT INTO students
    (student_id, name, course, email)
    VALUES (%s, %s, %s, %s)
    """

    values = (
        student_id,
        name,
        course,
        email
    )

    try:

        cursor.execute(query, values)

        connection.commit()
```

```
        messagebox.showinfo(
            "Success",
            "Student record inserted successfully."
        )

        clear_fields()

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:

        cursor.close()
        connection.close()
```

Step 8: Implement SELECT

Create the function:

```
def retrieve_students():

    text_area.delete("1.0", tk.END)

    connection = connect_database()
    cursor = connection.cursor()

    cursor.execute(
        "SELECT * FROM students"
    )

    records = cursor.fetchall()

    for record in records:

        text_area.insert(
            tk.END,
            "Student ID: " + str(record[0]) +
            "\nName: " + record[1] +
            "\nCourse: " + record[2] +
            "\nEmail: " + record[3] +
            "\n------------------------------\n"
        )

    cursor.close()
    connection.close()
```

Step 9: Implement UPDATE

Create the function:

```
def update_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    connection = connect_database()
    cursor = connection.cursor()

    query = """
    UPDATE students
    SET name=%s, course=%s, email=%s
    WHERE student_id=%s
    """

    values = (
        name,
        course,
        email,
        student_id
    )

    try:

        cursor.execute(query, values)

        connection.commit()

        if cursor.rowcount > 0:

            messagebox.showinfo(
                "Success",
                "Student record updated successfully."
            )

        else:

            messagebox.showwarning(
                "Warning",
                "Student ID not found."
            )

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:

        cursor.close()
        connection.close()
```

```
    clear_fields()
```

Step 10: Implement DELETE

Create the function:

```
def delete_student():

    student_id = id_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    connection = connect_database()
    cursor = connection.cursor()

    query = """
    DELETE FROM students
    WHERE student_id=%s
    """

    try:

        cursor.execute(
            query,
            (student_id,)
        )

        connection.commit()

        if cursor.rowcount > 0:

            messagebox.showinfo(
                "Success",
                "Student record deleted successfully."
            )

        else:

            messagebox.showwarning(
                "Warning",
                "Student ID not found."
            )

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:
```

```
        cursor.close()
        connection.close()

    clear_fields()
```

Step 11: Create Clear Function

```
def clear_fields():

    id_entry.delete(0, tk.END)
    name_entry.delete(0, tk.END)
    course_entry.delete(0, tk.END)
    email_entry.delete(0, tk.END)
```

Step 12: Create the GUI Window

Add:

```
root = tk.Tk()

root.title("Student Management System")

root.geometry("650x600")

root.resizable(False, False)
```

Step 13: Add Heading

```
heading = tk.Label(
    root,
    text="Student Management System",
    font=("Arial", 20, "bold")
)

heading.pack(pady=20)
```

Step 14: Create Input Fields

```
form_frame = tk.Frame(root)

form_frame.pack(pady=10)


tk.Label(
    form_frame,
    text="Student ID:",
    font=("Arial", 12)
).grid(row=0, column=0, padx=10, pady=10)
```

```
id_entry = tk.Entry(
    form_frame,
    width=30
)

id_entry.grid(row=0, column=1)


tk.Label(
    form_frame,
    text="Name:",
    font=("Arial", 12)
).grid(row=1, column=0, padx=10, pady=10)

name_entry = tk.Entry(
    form_frame,
    width=30
)

name_entry.grid(row=1, column=1)


tk.Label(
    form_frame,
    text="Course:",
    font=("Arial", 12)
).grid(row=2, column=0, padx=10, pady=10)

course_entry = tk.Entry(
    form_frame,
    width=30
)

course_entry.grid(row=2, column=1)


tk.Label(
    form_frame,
    text="Email:",
    font=("Arial", 12)
).grid(row=3, column=0, padx=10, pady=10)

email_entry = tk.Entry(
    form_frame,
    width=30
)

email_entry.grid(row=3, column=1)
```

Step 15: Create Buttons

```
button_frame = tk.Frame(root)

button_frame.pack(pady=15)


tk.Button(
```

```
    button_frame,
    text="INSERT",
    width=12,
    command=insert_student
).grid(row=0, column=0, padx=5)


tk.Button(
    button_frame,
    text="DISPLAY",
    width=12,
    command=retrieve_students
).grid(row=0, column=1, padx=5)


tk.Button(
    button_frame,
    text="UPDATE",
    width=12,
    command=update_student
).grid(row=0, column=2, padx=5)


tk.Button(
    button_frame,
    text="DELETE",
    width=12,
    command=delete_student
).grid(row=1, column=0, padx=5, pady=10)


tk.Button(
    button_frame,
    text="CLEAR",
    width=12,
    command=clear_fields
).grid(row=1, column=1, padx=5, pady=10)
```

Step 16: Create the Display Area

```
text_area = tk.Text(
    root,
    width=70,
    height=15
)

text_area.pack(pady=10)
```

Step 17: Run the Application

Finally:

```
root.mainloop()
```

Run:

```
student_mysql.py
```

Complete Program

For students who want the complete code in one place:

```
import tkinter as tk
from tkinter import messagebox
import mysql.connector


# ---------------- DATABASE CONNECTION ----------------

def connect_database():

    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="your_password",
        database="college_db"
    )

    return connection


# ---------------- INSERT ----------------

def insert_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "" or name == "" or course == "" or email == "":
        messagebox.showwarning(
            "Warning",
            "Please enter all details."
        )
        return

    try:

        connection = connect_database()
        cursor = connection.cursor()

        query = """
        INSERT INTO students
        (student_id, name, course, email)
        VALUES (%s, %s, %s, %s)
        """

        values = (
            student_id,
            name,
```

```
            course,
            email
        )

        cursor.execute(query, values)

        connection.commit()

        messagebox.showinfo(
            "Success",
            "Student record inserted successfully."
        )

        clear_fields()

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:

        cursor.close()
        connection.close()


# ---------------- SELECT ----------------

def retrieve_students():

    text_area.delete("1.0", tk.END)

    try:

        connection = connect_database()
        cursor = connection.cursor()

        cursor.execute(
            "SELECT * FROM students"
        )

        records = cursor.fetchall()

        if len(records) == 0:

            text_area.insert(
                tk.END,
                "No records found."
            )

        else:

            for record in records:

                text_area.insert(
                    tk.END,
                    "Student ID: " + str(record[0]) +
                    "\nName: " + record[1] +
                    "\nCourse: " + record[2] +
```

```
                    "\nEmail: " + record[3] +
                    "\n-----------------------------\n"
                )

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:

        cursor.close()
        connection.close()


# ---------------- UPDATE ----------------

def update_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    try:

        connection = connect_database()
        cursor = connection.cursor()

        query = """
        UPDATE students
        SET name=%s, course=%s, email=%s
        WHERE student_id=%s
        """

        values = (
            name,
            course,
            email,
            student_id
        )

        cursor.execute(query, values)

        connection.commit()

        if cursor.rowcount > 0:

            messagebox.showinfo(
                "Success",
                "Student record updated successfully."
            )
```

```
        else:

            messagebox.showwarning(
                "Warning",
                "Student ID not found."
            )

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:

        cursor.close()
        connection.close()

    clear_fields()


# ---------------- DELETE ----------------

def delete_student():

    student_id = id_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    try:

        connection = connect_database()
        cursor = connection.cursor()

        query = """
        DELETE FROM students
        WHERE student_id=%s
        """

        cursor.execute(
            query,
            (student_id,)
        )

        connection.commit()

        if cursor.rowcount > 0:

            messagebox.showinfo(
                "Success",
                "Student record deleted successfully."
            )

        else:
```

```
            messagebox.showwarning(
                "Warning",
                "Student ID not found."
            )

    except mysql.connector.Error as error:

        messagebox.showerror(
            "Error",
            str(error)
        )

    finally:

        cursor.close()
        connection.close()

    clear_fields()


# ---------------- CLEAR ----------------

def clear_fields():

    id_entry.delete(0, tk.END)
    name_entry.delete(0, tk.END)
    course_entry.delete(0, tk.END)
    email_entry.delete(0, tk.END)


# ---------------- GUI ----------------

root = tk.Tk()

root.title("Student Management System")

root.geometry("650x600")

root.resizable(False, False)


# Heading

heading = tk.Label(
    root,
    text="Student Management System",
    font=("Arial", 20, "bold")
)

heading.pack(pady=20)


# Form

form_frame = tk.Frame(root)

form_frame.pack(pady=10)


tk.Label(
```

```
    form_frame,
    text="Student ID:",
    font=("Arial", 12)
).grid(row=0, column=0, padx=10, pady=10)

id_entry = tk.Entry(
    form_frame,
    width=30
)

id_entry.grid(row=0, column=1)


tk.Label(
    form_frame,
    text="Name:",
    font=("Arial", 12)
).grid(row=1, column=0, padx=10, pady=10)

name_entry = tk.Entry(
    form_frame,
    width=30
)

name_entry.grid(row=1, column=1)


tk.Label(
    form_frame,
    text="Course:",
    font=("Arial", 12)
).grid(row=2, column=0, padx=10, pady=10)

course_entry = tk.Entry(
    form_frame,
    width=30
)

course_entry.grid(row=2, column=1)


tk.Label(
    form_frame,
    text="Email:",
    font=("Arial", 12)
).grid(row=3, column=0, padx=10, pady=10)

email_entry = tk.Entry(
    form_frame,
    width=30
)

email_entry.grid(row=3, column=1)


# Buttons

button_frame = tk.Frame(root)

button_frame.pack(pady=15)
```

```
tk.Button(
    button_frame,
    text="INSERT",
    width=12,
    command=insert_student
).grid(row=0, column=0, padx=5)


tk.Button(
    button_frame,
    text="DISPLAY",
    width=12,
    command=retrieve_students
).grid(row=0, column=1, padx=5)


tk.Button(
    button_frame,
    text="UPDATE",
    width=12,
    command=update_student
).grid(row=0, column=2, padx=5)


tk.Button(
    button_frame,
    text="DELETE",
    width=12,
    command=delete_student
).grid(row=1, column=0, padx=5, pady=10)


tk.Button(
    button_frame,
    text="CLEAR",
    width=12,
    command=clear_fields
).grid(row=1, column=1, padx=5, pady=10)


# Display area

text_area = tk.Text(
    root,
    width=70,
    height=15
)

text_area.pack(pady=10)


# Run application

root.mainloop()
```

Expected Operations

Students can test the application in this order:

1. INSERT

Enter:

```
Student ID: 101
Name: Rahul
Course: BSc Computer Applications
Email: rahul@gmail.com
```

Click **INSERT**.

2. DISPLAY

Click **DISPLAY**. The saved record will appear in the text area.

3. UPDATE

Enter Student ID `101`, change the Name/Course/Email, and click **UPDATE**.

4. DELETE

Enter Student ID `101` and click **DELETE**.

5. DISPLAY

Click **DISPLAY** again to verify that the record has been removed.

This gives students direct practice with **Python GUI → MySQL connection → SQL command → database → GUI output**, which matches the practical requirement closely.